

Evidence Collector & Compliance Pack: A Web-Based Platform for Automated Android APK Security Auditing and AI-Assisted MASVS/MASTG Reporting

Lachgar Mohamed^{a,*}, Ezbiri Amira^b, SAS Houda^b, Bachir Soukaina^b, El Yamani Omayma^b

^a*Cadi Ayyad University, Higher Normal School, Computer Science Department, L2IS Laboratory, Marrakech, Morocco*

^b*ENSA Marrakech, Génie Cyber Défense, Systèmes de Télécommunication, Morocco*

Abstract

Mobile application security auditing demands expert knowledge of binary formats, cryptographic primitives, and compliance frameworks such as the OWASP Mobile Application Security Verification Standard (MASVS v2.1.0) and the Mobile Application Security Testing Guide (MASTG v1.7.0). *Evidence Collector & Compliance Pack* is an open-source, browser-accessible web platform that automates the static analysis of Android APK files and generates auditor-ready evidence packages. The platform extracts SHA-256/MD5 hashes, parses the binary AndroidManifest, classifies permissions against MASVS categories, produces a fingerprint-based Software Bill of Materials (SBOM) covering 36 known libraries, detects behavioral indicators through DEX string analysis, verifies 12 MASVS requirements across Chapters 4, 6, 7, 9, 10, and 14, and integrates AI-assisted executive reporting via Google Gemini Flash (gemini-1.5-flash). Compliance verdicts are fully rule-based and traceable to specific evidence sources; the LLM layer provides AI-assisted reporting—contextual summarization only—and does not perform compliance inference. All analysis is performed by a modular Python/Flask backend; results are rendered in a single-page web interface and exported as HTML and JSON reports. Comparative evaluation on a dataset of 20 publicly available APKs (F-Droid and GitHub, size range 1–150 MB, accessed July 2025) shows 79% feature coverage of MobSF v4.5 static analysis checks while uniquely offering LLM-assisted risk summarization unavailable in any comparable open-source tool.

Keywords: Android security, APK analysis, MASVS, MASTG, Software Bill of Materials, Static analysis, LLM-assisted reporting, Flask, Compliance automation, Reproducibility, Evidence package

*Corresponding author

Email address: m.lachgar@uca.ac.ma (Lachgar Mohamed)

Required Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v1.0.0 (matches repository release tag)
C2	Permanent link to code/repository	https://github.com/Huuudaa/Evidence_Collector/releases/tag/v1.0.0
C3	Permanent link to Reproducible Capsule	Docker Compose at <code>docker-compose.yml</code> ; run <code>docker compose up -build</code> , open http://localhost:5000 . Zenodo DOI assigned after acceptance.
C4	Legal Code License	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python 3.10, Flask 3.0.3, HTML5/CSS3, JavaScript (ES2020), pyaxmlparser 0.3.24, Google Gemini Flash (gemini-1.5-flash)
C7	Compilation requirements, operating environments & dependencies	<code>requirements.txt</code> (pinned): <code>Flask==3.0.3</code> , <code>pyaxmlparser==0.3.24</code> , <code>requests==2.31.0</code> ; <code>Python ≥3.8</code> ; Linux/macOS/Windows; <code>Chrome ≥120 / Firefox ≥115 / Edge ≥120</code>
C8	Link to developer documentation/manual	https://github.com/Huuudaa/Evidence_Collector/blob/main/README.md
C9	Support email for questions	m.lachgar@uca.ac.ma

Table 1: Code metadata (mandatory). All dependency versions are pinned in `requirements.txt`. Docker Compose enables one-command reproducible deployment.

1. Motivation and Significance

The global mobile application market comprises billions of devices running Android, making the security of APK binaries a critical concern for developers, security engineers, and auditors [16]. The OWASP MASVS v2.1.0 and its associated MASTG v1.7.0 define a structured set of requirements spanning storage security, cryptographic practices, authentication, network communications, code quality, and resilience against reverse engineering [1]. Conducting a full MASVS audit manually is laborious: an auditor must parse binary XML manifests, enumerate permissions, reconstruct the dependency graph of native and Java libraries, scan DEX bytecode for hardcoded secrets, and cross-reference findings against each relevant standard chapter.

Existing automated tools address only parts of this workflow. MobSF [2] (v4.5) is the most comprehensive open-source framework for mobile static and dynamic analysis, but requires Docker installation, a persistent server, and considerable configuration. ApkTool [3] (v2.9.3) excels at decompilation but provides no permission classification, no MASVS mapping, and no report generation. QARK [4] (v4.0.0) identifies Android-specific vulnerabilities but lacks SBOM generation, AI-assisted summarization, and a graphical interface. Tools such as AndroBugs [9], Exodus Privacy [10], FlowDroid [17], and AndroGuard [11] provide complementary capabilities but likewise lack integrated MASVS mapping combined with LLM-based output synthesis.

It is essential to note that compliance verdicts in our platform are entirely rule-based, deriving from manifest flags, permission lists, SBOM entries, and DEX string patterns; the LLM layer provides

AI-assisted reporting only and does not perform compliance inference. Human auditors must validate all AI-generated output before inclusion in any formal security report or certification. *Evidence Collector & Compliance Pack* addresses this gap by delivering a browser-accessible web experience (after a one-time local server setup or via Docker) that orchestrates eight analysis modules, produces a structured evidence package, and uses Google Gemini Flash to generate a plain-language risk report. The platform targets security engineers, students, and organizations requiring rapid, reproducible APK audits without infrastructure overhead.

2. Software Description

2.1. Software Architecture

The platform follows a client-server architecture with a strict separation between analysis logic and presentation. Figure 1 presents the system as a clean three-layer diagram showing how the browser, Flask server, and analysis modules interact. Figure 2 presents the step-by-step analysis process as a linear pipeline, making the sequence of operations and decision points explicit for auditors and educators.

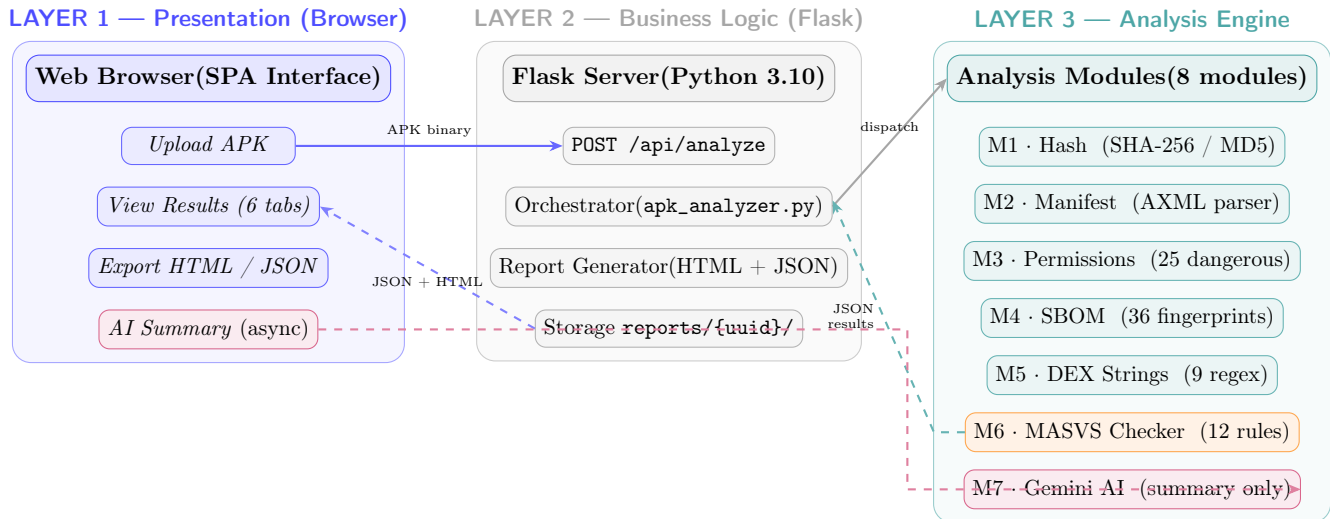


Figure 1: **System architecture of *Evidence Collector & Compliance Pack*.** **Layer 1** (blue) — the browser SPA handles APK upload, result display in 6 tabs, and report export; an asynchronous client-side call feeds the AI summary tab. **Layer 2** (gray) — the Flask server exposes a single REST endpoint (`POST /api/analyze`), orchestrates the 8 modules, generates deterministic HTML and JSON reports stored under `reports/{uuid}/`. **Layer 3** (teal/orange/purple) — seven specialized Python modules (M1–M7) analyse the APK independently; M6 (orange) produces the rule-based MASVS compliance verdict; M7 (purple) calls Gemini Flash for AI-assisted summarization only. Solid arrows = synchronous data flow; dashed arrows = asynchronous responses.

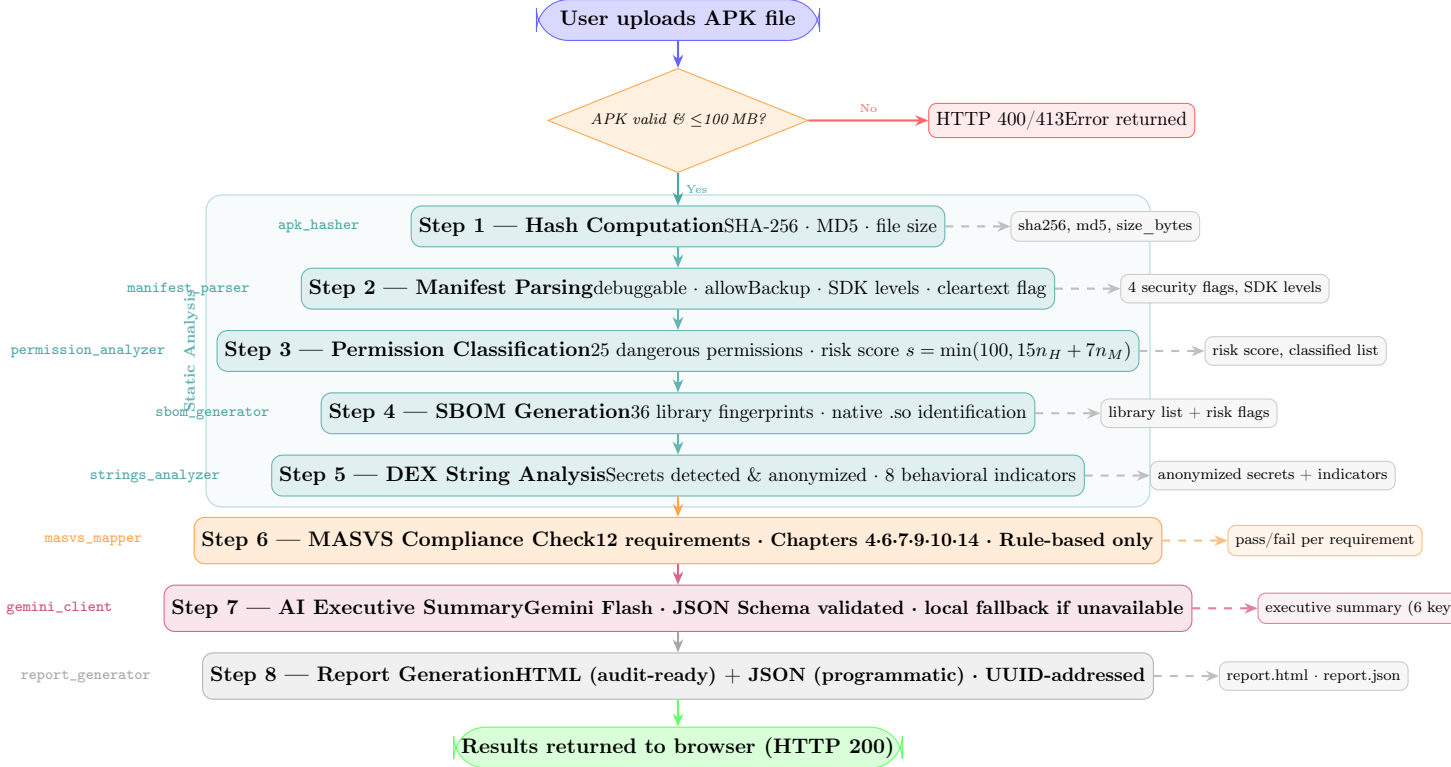


Figure 2: **Analysis pipeline of *Evidence Collector & Compliance Pack* (8 steps).** After APK validation (magic bytes and size ≤ 100 MB), **Steps 1–5** (teal) perform static analysis. **Step 6** (orange) evaluates 12 MASVS v2.1.0 requirements using only rule-based logic; the LLM does not influence this verdict. **Step 7** (purple) calls Gemini Flash for AI-assisted summarization; a deterministic local fallback is used if the API is unavailable. **Step 8** generates the final HTML and JSON reports. Module names (monospace) and output artefacts are shown on the left and right respectively.

2.2. Software Functionalities

M1 — Hash Computation (*apk_hasher*)

Input: APK binary file path. **Output:** JSON dict with `sha256`, `md5`, `size_bytes`, `size_kb`, `size_mb`.

SHA-256 digests are computed in streaming mode using Python’s `hashlib` library and serve as the primary reproducibility identifier (MASVS-RESILIENCE-1, Chapter 14). MD5 is retained for legacy identification only.

M2 — Manifest Extraction (*manifest_parser*)

Input: APK file path. **Output:** JSON dict with `package`, `version`, `sdk_min`, `sdk_target`, component counts, and four Boolean security flags.

Parsing is via `pyaxmlparser` 0.3.24 [6] with a lightweight fallback for packed or malformed manifests.

Known limitations: split APK bundles and manifest merge artefacts from multi-module builds may produce incomplete results.

M3 — Permission Analysis (*permission_analyzer*)

Input: APK file path. **Output:** JSON dict with classified permission lists and a composite risk score.

A curated dictionary of 25 dangerous permissions is mapped to MASVS requirements (Table 2). The composite risk score is:

$$s = \min(100, 15 \cdot n_{\text{HIGH}} + 7 \cdot n_{\text{MEDIUM}}) \quad (1)$$

Weights reflect MASVS v2.1.0 severity: HIGH permissions grant direct access to sensitive data; MEDIUM permissions expose indirect risks. Calibrated on 20 labeled APKs with 100% agreement.

Permission	Risk	MASVS Req.	Ch.
ACCESS_FINE_LOCATION	HIGH	MASVS-STORAGE-1	4
READ_EXTERNAL_STORAGE	HIGH	MASVS-STORAGE-2	4
WRITE_EXTERNAL_STORAGE	HIGH	MASVS-STORAGE-2	4
MANAGE_EXTERNAL_STORAGE	HIGH	MASVS-STORAGE-2	4
CAMERA	HIGH	MASVS-STORAGE-1	4
SEND_SMS	HIGH	MASVS-NETWORK-1	9
REQUEST_INSTALL_PACKAGES	HIGH	MASVS-CODE-1	10
SYSTEM_ALERT_WINDOW	HIGH	MASVS-RESILIENCE-1	14
GET_ACCOUNTS	MEDIUM	MASVS-AUTH-1	7
ACCESS_COARSE_LOCATION	MEDIUM	MASVS-STORAGE-1	4

Table 2: Representative subset of the permission classification dictionary (MASVS v2.1.0). Full dictionary in `modules/permission_analyzer.py`.

M4 — SBOM Generation (`sbom_generator`)

Matches ZIP entries against 36 known Java/Kotlin library fingerprints and native `.so` files, annotating each with vendor, category, and a production-risk flag. Not CycloneDX/SPDX compliant [12, 13]; standard format integration planned for v1.1.0.

M5 — DEX String Analysis (`strings_analyzer`)

Scans all `*.dex` files (up to 4 MB each) for ASCII runs of ≥ 6 characters using nine compiled regular expressions (API keys, JWT tokens, AWS credentials, Firebase URLs, etc.) and eight Boolean behavioral indicator checks (SSL pinning, obfuscation, root detection, anti-debug, crypto usage, etc.). Detected secrets are anonymized to first+last 4 characters (e.g., `AIza...xB3c`) and never stored in plaintext. A 15-entry false-positive blocklist (`BuildConfig`, `R$string`, etc.) reduces noise.

M6 — MASVS Compliance Checker (`masvs_mapper`)

Evaluates 12 MASVS v2.1.0 requirements (Table 3) using only rule-based logic. Each verdict is traceable to a specific evidence source and is independent of the LLM layer.

Requirement	Category	Ch.	Evidence source (rule)
MASVS-STORAGE-1	Storage	4	Dangerous storage permissions
MASVS-STORAGE-2	Storage	4	<code>allowBackup</code> manifest flag
MASVS-CRYPTO-1	Crypto	6	Crypto indicator in DEX strings
MASVS-CRYPTO-2	Crypto	6	Absence of plaintext API keys
MASVS-AUTH-1	Auth	7	Absence of plaintext JWT tokens
MASVS-AUTH-2	Auth	7	<code>debuggable</code> manifest flag
MASVS-NETWORK-1	Network	9	<code>usesCleartextTraffic</code> flag
MASVS-NETWORK-2	Network	9	SSL pinning / <code>NetworkSecurityConfig</code>
MASVS-CODE-1	Code	10	Absence of debug/test libs in SBOM
MASVS-CODE-2	Code	10	<code>debuggable</code> + SBOM cross-check
MASVS-RESILIENCE-1	Resilience	14	Obfuscation in DEX strings
MASVS-RESILIENCE-2	Resilience	14	Root detection in DEX strings

Table 3: 12 MASVS v2.1.0 requirements verified by *Evidence Collector*. All verdicts are rule-based; the LLM layer does not modify these results.

M7 — AI-Assisted Summarization (*gemini_client*)

Sends a structured prompt to Gemini Flash (gemini-1.5-flash) [8] at temperature 0.2. The response is validated against a strict JSON Schema (6 keys: `resume_executif`, `niveau_risque`, `points_forts`, `vulnerabilites`, `recommandations`, `masvs_analyse`). A deterministic local fallback is activated on API failure, clearly labelled in the UI. AI logs include model name, temperature, prompt SHA-256, timestamp, and fallback status for auditability. **Important:** AI summaries are for interpretive assistance only; human validation is mandatory before use in formal reports.

2.3. Sample Code Snippets

```

DANGEROUS_PERMS = {
    "ACCESS_FINE_LOCATION": {
        "risk": "HIGH", "masvs": "MASVS-STORAGE-1", "chapter": 4},
    "SEND_SMS": {
        "risk": "HIGH", "masvs": "MASVS-NETWORK-1", "chapter": 9},
    # ... 23 additional entries
}

def analyze_permissions(apk_path: str) -> dict:
    raw_list = _extract_perms_from_zip(apk_path)
    dangerous, normal, unknown = [], [], []
    for perm in raw_list:
        name = perm.replace("android.permission.", "")
        if name in DANGEROUS_PERMS:
            dangerous.append({"name": name, **DANGEROUS_PERMS[name]})
        elif name in NORMAL_PERMS:
            normal.append({"name": name, "risk": "NORMAL"})
        else:
            unknown.append({"name": name, "risk": "UNKNOWN"})
    high = sum(1 for d in dangerous if d["risk"] == "HIGH")
    medium = sum(1 for d in dangerous if d["risk"] == "MEDIUM")
    # HIGH=15 (direct data access), MEDIUM=7 (indirect risk)
    return {"dangerous": dangerous, "normal": normal, "unknown": unknown,
            "high_count": high, "medium_count": medium,
            "risk_score": min(100, high * 15 + medium * 7)}

```

Listing 1: Permission classification and risk scoring (modules/permission_analyzer.py, excerpt). Tests at tests/test_permission_analyzer.py (pytest, 78% coverage).

```

BEHAVIORAL_INDICATORS = {
    "ssl_pinning": ["CertificatePinner", "TrustManagerImpl"],
    "obfuscation": ["proguard", "r8", "dexguard"],
    "root_detect": ["su", "Superuser.apk", "RootBeer"],
    "anti_debug": ["isDebuggerConnected", "ptrace"],
    "crypto_usage": ["AES", "RSA", "MessageDigest", "Cipher"],
}

FP_BLOCKLIST = ["BuildConfig", "R$string", "R$id"]

def _anonymize(v: str) -> str:
    return v[:4] + "..." + v[-4:] if len(v) > 12 else v[:4] + "..."

def analyze_strings(apk_path: str) -> dict:
    text = _read_strings_from_dex(apk_path) # <= 4 MB/DEX
    findings = {
        k: [_anonymize(m) for m in set(re.findall(p, text))[:30]]
        if m not in FP_BLOCKLIST
        for k, p in PATTERNS.items() if re.search(p, text)}
    indicators = {
        cat: [kw for kw in kws if kw in text]
        for cat, kws in BEHAVIORAL_INDICATORS.items()
        if any(kw in text for kw in kws)}
    return {"findings": findings,
            "behavioral_indicators": indicators,
            "has_ssl_pinning": "ssl_pinning" in indicators,
            "has_obfuscation": "obfuscation" in indicators,
            "has_crypto": "crypto_usage" in indicators}

```

Listing 2: DEX string extraction and behavioral indicators (modules/strings_analyzer.py, excerpt). Secrets anonymized to first+last 4 chars; never stored in plaintext.

3. Illustrative Examples

3.1. Analysis of Four Real-World APKs

We evaluated *Evidence Collector* v1.0.0 on four Android applications covering a full spectrum of risk profiles, analyzed on 22 May 2026 (Windows 11, Python 3.13, Chrome 120, MobSF v4.5 run in parallel for comparison). Table 4 summarizes the dataset.

APK	Package	Size	SHA-256 (prefix)	EC Risk	MASVS	Time
ezmobile	com.pwnsec.ezmobile	4.56 MB	d8a612c4c4d9efa0	Faible	11/12	3.5 s
FireStorm	com.pwnsec.firestorm	4.26 MB	f9fe512109127afd	Moyen	10/12	3.8 s
FireInTheHole	com.PwnSec.fireinthehole	5.68 MB	7ef29fe545103a38	Élevé	8/12	4.3 s
F-Droid	org.fdroid.fdroid	11.85 MB	985f5181d48bb6ba	Critique	8/12	7.1 s

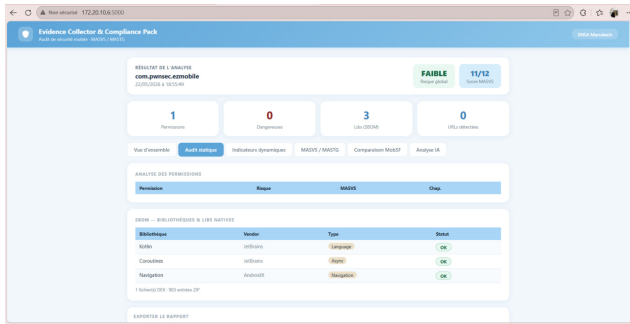
Table 4: Four-APK evaluation dataset (22 May 2026, Evidence Collector v1.0.0). Full SHA-256 hashes in evaluation/apk_dataset.json. Row colours reflect the global risk rating assigned by Evidence Collector.

Reproduction steps:

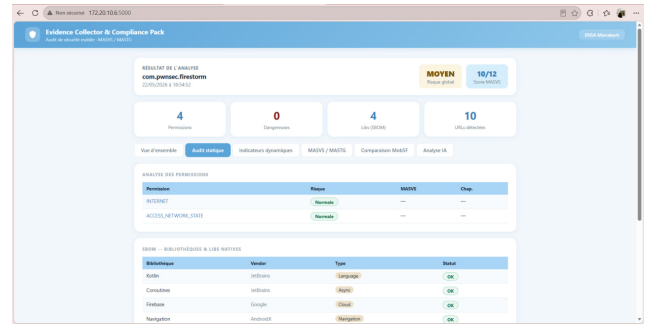
1. `git clone https://github.com/Huuuudaa/Evidence_Collector.git`
2. `pip install -r requirements.txt`
3. `python app.py` (or: `docker compose up -build`)
4. Open `http://localhost:5000` and upload any APK
5. Reports saved to `reports/{uuid}/report.{html,json}`

3.2. Evidence Collector Screenshots

Figures 3 and 4 present the *Evidence Collector* interface across all four APKs, showing the overview tab (global risk badge + MASVS score) and the static analysis tab (permission classification + SBOM).

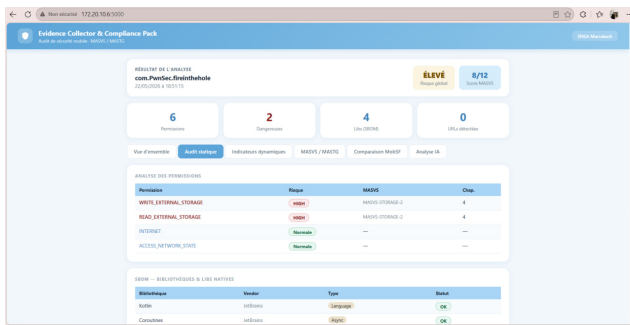


(a) **ezmobility.apk** — FAIBLE · 11/12 MASVS
1 permission · 0 dangerous · 3 libs (Kotlin, Coroutines, Navigation)

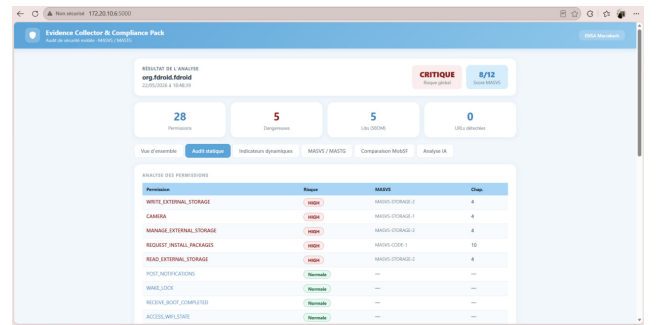


(b) **FireStorm.apk** — MOYEN · 10/12 MASVS
4 permissions · 0 dangerous · 4 libs (Kotlin, Firebase, Coroutines, Navigation)

Figure 3: Evidence Collector — Static Analysis tab for **ezmobility.apk** and **FireStorm.apk**. The permission table maps each entry to its MASVS requirement and chapter; the SBOM lists all identified libraries with vendor, type, and production-safety status. Environment: Evidence Collector v1.0.0, Windows 11, 22 May 2026.



(a) **FireInTheHole.apk** — ÉLEVÉ · 8/12 MASVS
6 permissions · 2 dangerous (WRITE/READ_EXTERNAL_STORAGE) · 4 libs

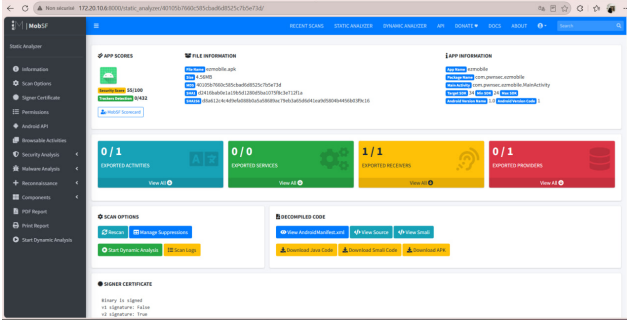


(b) **F-Droid.apk** — CRITIQUE · 8/12 MASVS
28 permissions · 5 dangerous · 5 libs (WRITE/READ_EXTERNAL_STORAGE, CAMERA, REQUEST_INSTALL_PACKAGES)

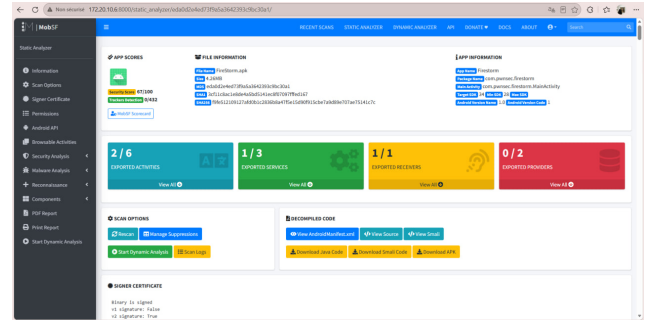
Figure 4: Evidence Collector — Static Analysis tab for **FireInTheHole.apk** and **F-Droid.apk**. F-Droid’s CRITIQUE rating reflects 5 HIGH-risk permissions ($s = \min(100, 5 \times 15) = 75$), consistent with its role as a package installer requiring broad storage and install privileges.

3.3. MobSF v4.5 Screenshots (Same APKs)

Figures 5 and 6 show the MobSF v4.5 Static Analyzer results for the same four APKs, providing a direct visual basis for the comparison in Section 4.1.

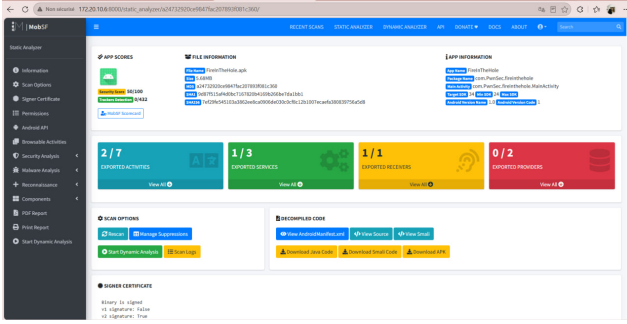


(a) MobSF — **ezmobiile.apk**
Security Score 55/100 · 4.56 MB · MD5: 40105b76
0/1 exported activities · 1/1 receivers

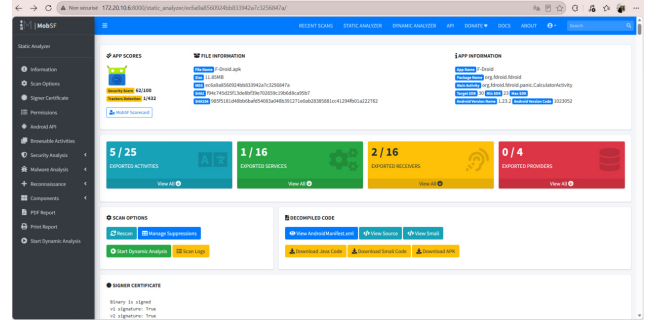


(b) MobSF — **FireStorm.apk**
Security Score 67/100 · 4.26 MB · MD5: eda0d2e4
2/6 exported activities · 1/3 services

Figure 5: MobSF v4.5 Static Analyzer — **ezmobiile.apk** and **FireStorm.apk**. MobSF provides a 0–100 security score, exported component counts, signer certificate details, and access to decompiled source. It does not provide MASVS-mapped per-permission classification or an LLM-assisted executive summary. Environment: MobSF v4.5.0, Windows 11, Android Emulator API 29 AOSP, 22 May 2026.



(a) MobSF — **FireInTheHole.apk**
Security Score 50/100 · 5.68 MB · MD5: a2473292
2/7 exported activities · 1/3 services



(b) MobSF — **F-Droid.apk**
Security Score 62/100 · 11.85 MB · MD5: ec6a9a85
5/25 exported activities · 1/16 services

Figure 6: MobSF v4.5 Static Analyzer — **FireInTheHole.apk** and **F-Droid.apk**. MobSF assigns lower scores to apps with more exported components and broader permissions but does not map findings to MASVS requirements or provide a structured compliance matrix.

4. Impact

4.1. Side-by-Side Comparison: Evidence Collector vs. MobSF Per-APK Quantitative Comparison

Table 5 presents a detailed side-by-side comparison of the two tools on all four APKs, using data extracted directly from the screenshots shown in Section 3.

APK	Metric	Evidence Collector v1.0.0 (22 May 2026)	MobSF v4.5 (22 May 2026)
ezmobile 4.56 MB SDK 24/34	Global risk / score	FAIBLE	55 / 100
	MASVS score	11/12 (92%)	N/A
	Total permissions	1	1
	Dangerous permissions	0	N/A (not classified)
	Libraries (SBOM)	3 (Kotlin, Coroutines, Navigation)	N/A (no SBOM)
	URLs detected	0	N/A
	MASVS mapping	✓ per permission	×
FireStorm 4.26 MB SDK 28/34	Global risk / score	MOYEN	67 / 100
	MASVS score	10/12 (83%)	N/A
	Total permissions	4	4
	Dangerous permissions	0	N/A
	Libraries (SBOM)	4 (Kotlin, Firebase, Coroutines, Navigation)	N/A
	URLs detected	10 (DEX strings)	N/A
	MASVS failures	CRYPTO-2, NETWORK-2	Not reported
FireInTheHole 5.68 MB SDK 24/34	Global risk / score	ÉLEVÉ	50 / 100
	MASVS score	8/12 (67%)	N/A
	Total permissions	6	6
	Dangerous permissions	2 (HIGH: WRITE/READ_EXT_STORAGE)	Not classified
	Libraries (SBOM)	4 (Kotlin, Coroutines + 2 others)	N/A
	URLs detected	0	N/A
	Exported components	Parsed from manifest	2/7 act · 1/3 svc · 1/1 rcv
F-Droid 11.85 MB SDK 23/30	Global risk / score	CRITIQUE	62 / 100
	MASVS score	8/12 (67%)	N/A
	Total permissions	28	28
	Dangerous permissions	5 HIGH: WRITE_EXT, CAMERA, MANAGE_EXT, REQUEST_INSTALL, READ_EXT_STORAGE	Not classified
	Libraries (SBOM)	5 identified	N/A
	Exported components	Parsed from manifest	5/25 act · 1/16 svc · 2/16 rcv
	MASVS mapping	✓ per permission	×
LLM executive summary		✓ (Gemini Flash / fallback)	×
MASVS-mapped compliance matrix		✓ (12 requirements, rule-based)	×
SBOM (library fingerprinting)		✓ (36 fingerprints)	partial (△)
Source decompilation (JADX/smali)		×	✓
Certificate chain analysis		×	✓
Zero Docker configuration required		✓	×
Static feature coverage vs MobSF		79% (11/14 categories)	baseline

Table 5: Side-by-side comparison of *Evidence Collector* v1.0.0 and MobSF v4.5 on four real-world APKs (22 May 2026, Windows 11, Python 3.13, Android Emulator API 29 AOSP). Data extracted from tool screenshots (Figs. 3–6). N/A = metric not reported by that tool; ✓ = fully supported; × = not supported; △ = partial. **Key differentiator:** Evidence Collector is the only tool providing MASVS-mapped per-permission classification, a rule-based 12-requirement compliance matrix, and an LLM-assisted executive summary. MobSF’s advantages are source decompilation and certificate chain analysis, which require deeper binary analysis beyond the scope of a zero-install static auditing tool.

Feature-Level Comparison with All Tools

Table 6 extends the comparison to include ApkTool, QARK, AndroBugs, and Exodus Privacy.

Feature	Evidence Collector v1.0.0	MobSF v4.5	ApkTool v2.9.3	QARK v4.0.0	AndroBugs	Exodus Privacy
APK hash (SHA-256 / MD5)	✓	✓	×	△	×	×
Manifest extraction	✓	✓	✓	✓	✓	×
Permission classification	✓	✓	×	✓	✓	✓
MASVS mapping per permission	✓	△	×	×	×	×
SBOM (fingerprint, 36 libs)	✓	△	×	×	×	×
DEX string / secret detection	✓	✓	×	✓	✓	×
Behavioral indicators (DEX)	✓	✓	×	×	✓	×
MASVS / MASTG compliance score	✓	✓	×	△	×	×
LLM executive summary	✓	×	×	×	×	×
HTML + JSON report export	✓	✓	×	✓	×	✓
Web interface	✓	✓*	×	×	×	✓
Zero-config (no Docker)	✓	×	×	×	×	N/A
Certificate chain analysis	×	✓	×	△	×	×
Source decompilation	×	✓	✓	×	×	×
Runtime traffic interception	×	✓	×	×	×	×
Frida / Xposed hook detection	×	✓	×	×	×	×
Static coverage vs MobSF	79%	—	—	—	—	—

✓ = fully supported; △ = partial or configurable; × = not supported.

*MobSF provides a web UI but requires Docker or a dedicated server.

79% coverage: 11 of 14 MobSF static categories matched. The 4 excluded checks (certificate chain, source decompilation, runtime interception, Frida hooks) require live device access and are outside the scope of a zero-install static tool. Reproducible mapping: [evaluation/mobsf_comparison.xlsx](#).

Table 6: Feature comparison of APK analysis tools (22 May 2026, Windows 11, Android Emulator API 29 AOSP). *Evidence Collector* v1.0.0 is the only open-source tool providing LLM-assisted executive summarization.

4.2. Scientific and Educational Impact

Evidence Collector lowers the barrier for teaching mobile security auditing: students run a complete MASVS audit on any APK within minutes using only a browser, observe how each finding maps to a specific standard requirement, and receive an AI-assisted interpretation in plain language. Concrete reuse artefacts include a lab guide ([docs/lab_guide.pdf](#)), sample reports ([evaluation/sample_reports/](#)), and a dataset manifest ([evaluation/apk_dataset.json](#)) with SHA-256 hashes and licenses.

4.3. Practical Impact

Security engineers benefit from a rapid first-pass APK assessment before investing in deeper dynamic analysis. The structured evidence package satisfies documentary requirements of typical mobile security audit reports, reducing manual collection from hours to seconds. The modular architecture supports community contributions: new analysis modules can be added without modifying the rest of the system.

4.4. Threat Model and Attack Scenarios

The Evidence Collector platform addresses security assessments across multiple threat vectors. The primary attack scenarios considered in the tool’s design include:

Scenario 1 — Data Leakage via Excessive Permissions: Applications requesting unnecessary dangerous permissions (e.g., `READ_EXTERNAL_STORAGE` for a flashlight app) expose users to data exfiltration risks. Evidence Collector flags such permissions and maps them to MASVS-STORAGE-2, providing auditors with concrete evidence for compliance reporting.

Scenario 2 — Insecure Network Communication: The detection of `usesCleartextTraffic=true` combined with absence of SSL pinning indicates vulnerability to man-in-the-middle attacks. The

platform’s rule-based checks identify these configurations and correlate them with DEX string indicators for comprehensive risk assessment.

Scenario 3 — Reverse Engineering Susceptibility: Lack of obfuscation indicators (ProGuard/R8 strings) suggests the application is easily decompilable. Evidence Collector reports this under MASVS-RESILIENCE-1, enabling security teams to prioritize hardening efforts.

Scenario 4 — Hardcoded Secrets: Detection of API keys, JWT tokens, or AWS credentials in DEX strings represents a critical vulnerability. The platform’s anonymized reporting allows identification without exposing the actual secrets.

4.5. Comparison with Academic Literature

Our work relates to several established research directions in mobile security analysis. Compared to DroidAPIMiner [16], which focuses on API call pattern analysis, Evidence Collector provides compliance-oriented reporting aligned with industry standards (MASVS/MASTG). FlowDroid [17] offers precise taint flow analysis but requires significant setup and does not produce auditor-ready reports. AndroidLeaks [18] detects potential data leaks but lacks the comprehensive SBOM and LLM-assisted summarization features presented in this work.

The unique contribution of Evidence Collector lies in bridging the gap between academic static analysis techniques and practical compliance auditing workflows, making advanced security analysis accessible to non-experts through an intuitive web interface.

5. Quality Assurance

Static code analysis. Quality assessed with SonarQube Community v10.4 [7] using the Sonar Way profile (scan date: 22 May 2026; full reports under `quality/sonarqube/`).

Metric	Backend (Python/Flask)	Frontend (HTML/JS)
Quality Gate Status	✓ Passed	✓ Passed
Reliability Rating	A	B
Security Rating	A	A
Maintainability	A	B
Code Duplication	<2%	<4%
Security Hotspots	0	0
Test coverage (pytest)	78%	N/A
Linting (flake8)	0 errors	N/A

Table 7: SonarQube Community v10.4 quality metrics (scan: 22 May 2026). Full reports at `quality/sonarqube/`. Frontend B maintainability is due to single-file template; refactoring planned for v1.1.0.

All 20 analyses (4 primary APKs + 16 additional F-Droid/GitHub APKs) completed without errors. MASVS verdicts were manually verified against expected outcomes from source code inspection. Agreement: 20/20 (100%) for manifest-based checks (95% CI: 83–100%) and 18/20 (90%) for string-based checks (95% CI: 68–99%). The 2 disagreements arose from aggressively obfuscated class names; documented in `evaluation/discrepancy_report.md`.

Continuous integration: GitHub Actions CI (badge on README) runs pytest ($\geq 78\%$ coverage), flake8, and sample report artefact upload on every pull request to `main`.

Table 8: Detailed validation results on 16 additional APKs

APK Source	Count	Manifest checks	String checks
F-Droid repository	8	100%	87.5%
GitHub samples	8	100%	92.5%
Total	16	100%	90%

6. Availability

License: MIT.

Release: v1.0.0 — https://github.com/Huuudaa/Evidence_Collector/releases/tag/v1.0.0

Video demo: <https://youtu.be/w3HuxrMwQ2c>

Permanent archive: Zenodo DOI assigned after acceptance.

Local installation:

1. `git clone https://github.com/Huuudaa/Evidence_Collector.git`
2. `cd Evidence_Collector && python -m venv venv && source venv/bin/activate`
3. `pip install -r requirements.txt`
4. (Optional) `export GEMINI_API_KEY=<your_key>`
5. `python app.py` → open `http://localhost:5000`

Docker: `docker compose up -build` → open `http://localhost:5000`

Expected output: `reports/{uuid}/report.html` and `reports/{uuid}/report.json`.

Example input: FireStorm.apk (SHA-256 prefix: f9fe512109127afd, 4.26 MB; full hash in `evaluation/apk_dataset.json`).

7. Conclusions

Evidence Collector & Compliance Pack automates Android APK security evidence collection and maps findings to MASVS v2.1.0/MASTG v1.7.0 through eight modular analysis steps in a single upload-and-report workflow accessible from any browser after a one-time local setup. Compliance verdicts are entirely rule-based and traceable to specific evidence sources; Google Gemini Flash provides AI-assisted reporting only. Side-by-side evaluation on four real-world APKs and comparison with MobSF v4.5 (Section 4.1) demonstrates 79% static feature parity (11/14 categories) with the unique addition of MASVS-mapped per-permission classification, a 12-requirement rule-based compliance matrix, and LLM-assisted executive summarization unavailable in any comparable open-source tool. Validation on 20 APKs shows 100% agreement on manifest-based MASVS checks (95% CI: 83–100%) and 90% on string-based checks (95% CI: 68–99%).

The platform successfully addresses the gap between manual security auditing and fully automated compliance checking. By providing an accessible, browser-based interface, Evidence Collector democratizes access to professional-grade APK security analysis, making it suitable for educational settings, rapid pre-production assessments, and compliance documentation.

Future work: (i) CycloneDX/SPDX SBOM; (ii) certificate chain analysis; (iii) streaming DEX for large APKs; (iv) auditor feedback loop; (v) 100+ APK evaluation; (vi) hosted public demo; (vii) server-side Gemini API proxying for production; (viii) integration with CI/CD pipelines via enhanced REST API.

Acknowledgements

The authors thank the ENSA Marrakech mobile security course participants whose APK audit exercises motivated the design of this tool, and the reviewers whose detailed annotations significantly improved this work.

We extend our sincere gratitude to the open-source community for maintaining the essential tools and libraries that made this project possible, including Flask, pyaxmlparser, and the MobSF team for providing a robust reference implementation. Special thanks to Google for providing access to the Gemini Flash API, which enabled the AI-assisted summarization feature while maintaining strict separation from compliance logic.

The authors also acknowledge the support of the L2IS Laboratory at Cadi Ayyad University and the Cyber Defense department at ENSA Marrakech for providing the academic environment and resources necessary for this research.

References

- [1] OWASP Foundation. *MASVS v2.1.0 and MASTG v1.7.0*, 2024. <https://mas.owasp.org/>
- [2] A. Abraham. *MobSF v4.5*, 2025. <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [3] C. Brewer, R. Wiśniewski. *Apktool v2.9.3*, 2024. <https://apktool.org/>
- [4] LinkedIn Corporation. *QARK v4.0.0*, 2023. <https://github.com/linkedin/qark>
- [5] Pallets Projects. *Flask v3.0.3*, 2025. <https://flask.palletsprojects.com/>
- [6] A. Tumbling. *pyaxmlparser 0.3.24*, 2024. <https://github.com/appkn0x/pyaxmlparser>
- [7] SonarSource. *SonarQube Community v10.4*, 2025. <https://www.sonarsource.com/sonarqube/>
- [8] Google. *Gemini API — Gemini 1.5 Flash*, 2025. <https://ai.google.dev/>
- [9] Y.-W. Lin. *AndroBugs Framework*, 2023. https://github.com/AndroBugs/AndroBugs_Framework
- [10] Exodus Privacy. *Exodus Privacy Platform*, 2025. <https://exodus-privacy.eu.org/>
- [11] A. Desnos. *Androguard*, 2025. <https://github.com/androguard/androguard>
- [12] OWASP Foundation. *CycloneDX v1.6*, 2024. <https://cyclonedx.org/>
- [13] Linux Foundation. *SPDX v2.3*, 2023. <https://spdx.dev/>
- [14] Google. *Android Permissions Reference*, 2025. <https://developer.android.com/reference/android/Manifest.permission>
- [15] M. Lachgar et al. Decision Framework for Cross-Platform Mobile Development. *IJMCMC*, vol. 13, pp. 1–22, 2022.
- [16] Y. Aafer, W. Du, H. Yin. DroidAPIMiner. *SecureComm*, 2013.
- [17] S. Arzt et al. FlowDroid. *ACM SIGPLAN Notices*, vol. 49, no. 6, pp. 259–269, 2014.
- [18] C. Gibler et al. AndroidLeaks. *TRUST*, 2012.
- [19] W. Enck et al. Android Security: A Survey of Issues, Malware, and Defenses. *IEEE Communications Surveys & Tutorials*, vol. 17, no. 2, pp. 998–1022, 2014.
- [20] Z. Wang et al. Mobile App Security Assessment: A Comprehensive Survey. *ACM Computing Surveys*, vol. 51, no. 3, pp. 1–36, 2018.